

Překážky pro Vikuňu

Vikuňa chce proklusat Altiplanem. S sebou má mapu v podobě čtvercové mřížky o velikosti $N \times M$ polí. Řádky jsou očíslované od 0 do $N - 1$ shora dolů, sloupce od 0 do $M - 1$ zleva doprava. Pole mapy v řádku i a sloupci j ($0 \leq i < N, 0 \leq j < M$) budeme značit (i, j) .

Jedná se o vikuňu vzdělanou, dopředu si totiž nastudovala klima regionu. Zjistila, že všechna pole v řádku vždy sdílí stejnou **teplotu** a že všechna pole v sloupci vždy sdílí stejnou **vlhkost**. Vikuňa vám předala dva seznamy celých čísel: T a H o délkách N a M . Zde $T[i]$ ($0 \leq i < N$) udává teplotu polí na řádku i a $H[j]$ ($0 \leq j < M$) značí vlhkost polí ve sloupci j .

Vikuňa je hrdou představitelkou pansofie, nastudovala si totiž i místní flóru a všimla si, že pole (i, j) je **nezarostlé** právě tehdy, když je jeho teplota vyšší než jeho vlhkost, formálně $T[i] > H[j]$.

Vikuňa umí cestovat po Altiplanu pouze po **platných cestách**. Platná cesta je taková posloupnost různých polí, která splňuje následující podmínky:

- Každá dvojice po sobě jdoucích polí v cestě sdílí stranu.
- Všechna pole cesty jsou nezarostlá.

Vaším úkolem je zodpovědět Q dotazů. Každý dotaz sestává ze čtyř celých čísel: L, R, S a D . Musíte rozhodnout, zda existuje platná cesta taková, že platí:

- Začíná na poli $(0, S)$ a končí polem $(0, D)$.
- Všechna pole cesty leží mezi sloupci L a R včetně.

Máte zaručeno, že obě pole $(0, S)$ a $(0, D)$ jsou nezarostlá.

Implementační detaily

První funkcí, kterou máte implementovat, je:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : pole délky N specifikující teplotu v každém řádku.
- H : pole délky M specifikující vlhkost v každém sloupci.
- Tato funkce je zavolána právě jednou pro každý vstup, a to před všemi voláními `can_reach`.

Druhou funkcí, kterou máte implementovat, je:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : celá čísla popisující dotaz.
- Tato funkce je zavolána Q krát pro každý vstup.

Funkce výše by měla vrátit `true` právě tehdy, pokud existuje platná cesta z pole $(0, S)$ do pole $(0, D)$ taková, že všechna pole cesty leží mezi sloupci L a R včetně.

Omezení

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ pro všechna i taková, že $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ pro všechna j taková, že $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Pole $(0, S)$ i $(0, D)$ jsou nezarostlá.

Podúlohy

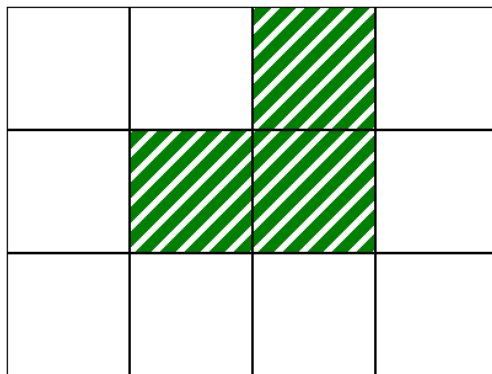
Podúloha	Počet bodů	Další omezení
1	10	$L = 0, R = M - 1$ pro každý dotaz. $N = 1$.
2	14	$L = 0, R = M - 1$ pro každý dotaz. $T[i - 1] \leq T[i]$ pro všechna i taková, že $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ pro každý dotaz. $N = 3$ a $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ pro každý dotaz. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ pro každý dotaz.
6	17	Žádná další omezení.

Příklad

Uvažujme následující volání:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

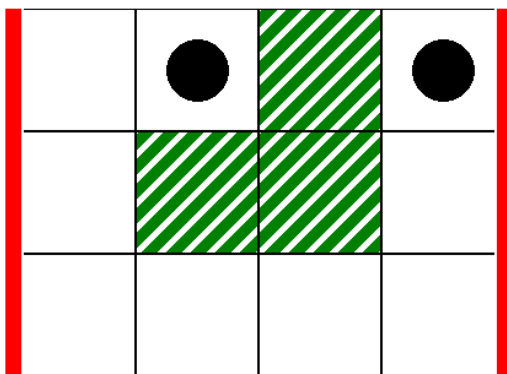
Toto volání odpovídá mapě na následujícím obrázku, kde bílá pole jsou nezarostlá:



Pro první dotaz uvažme následující volání:

```
can_reach(0, 3, 1, 3)
```

To odpovídá situaci na následujícím obrázku. Tlusté vertikální čáry značí rozmezí sloupců od $L = 0$ do $R = 3$. Černé kroužky značí počáteční a konečné pole.



V tomto případě se může vikuňa dostat z pole $(0, 1)$ do pole $(0, 3)$ po následující platné cestě:

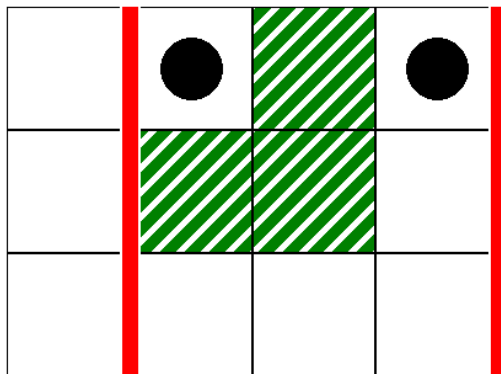
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Tudíž by toto volání mělo vrátit `true`.

Jakožto druhou otázku uvažme následující volání:

```
can_reach(1, 3, 1, 3)
```

To odpovídá situaci na následujícím obrázku.



V tomto případě neexistuje žádná platná cesta z pole $(0, 1)$ do pole $(0, 3)$ taková, že by všechna pole cesty ležela mezi sloupci 1 a 3 včetně. Tudíž by toto volání mělo vrátit `false`.

Ukázkový grader

Formát vstupu:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Zde $L[k]$, $R[k]$, $S[k]$ a $D[k]$ ($0 \leq k < Q$) udávají parametry pro každé zavolání funkce `can_reach`.

Formát výstupu:

```
A[0]
A[1]
...
A[Q-1]
```

Zde $A[k]$ ($0 \leq k < Q$) je 1 pokud volání `can_reach(L[k], R[k], S[k], D[k])` vrátilo `true`, v opačném případě 0.